IIS5008 Hardware Security
Final Project
Design of ECC Systems for Reliable Data Transmission
(Due:2026/06/19 23:59:59)

Introduction

As modern computing systems increasingly rely on non-volatile memory technologies such as NAND Flash, ensuring data reliability has become a critical challenge. Unlike idealized environments where errors are random and independent, real-world storage systems are affected by complex and structured error patterns, including read disturb, program interference, retention errors, and erase failures.

Error-Correcting Codes (ECC) are widely used to protect data integrity in storage systems. However, designing an effective ECC scheme involves balancing multiple trade-offs, including redundancy overhead, error tolerance, and computational efficiency. Furthermore, ECC schemes must be robust against diverse and evolving error conditions.

In this project, you will design and implement an **ECC-based data protection system** under a realistic NAND Flash error model. Instead of relying on simple random corruption, your system must handle structured and time-dependent errors inspired by real hardware behavior.

Background

1. Error-Correcting Codes (ECC)

   Error-correcting codes are algorithms that add structured redundancy to digital data to detect and correct errors that occur during storage or transmission. By encoding original data with additional parity or check symbols, ECC schemes allow the recovery of corrupted bits or symbols without the need for retransmission. In memory systems, ECC is critical for protecting against soft errors caused by cosmic rays, electrical noise, and other transient faults, thereby improving system reliability and uptime.

Problem Description

You are required to implement a system consisting of two components:

$$m \rightarrow \text{encode} \rightarrow c \rightarrow (\text{Flash Error Model}) \rightarrow c' \rightarrow \text{decode} \rightarrow m'$$

Where:

- m: original data (input)

- c: encoded data (stored in flash)
- c': corrupted data after flash errors
- m': decoded output

Your goal:

Design an **ECC scheme** such that:

- m' == m (exact recovery) under error conditions
- Redundancy is minimized
- Runtime is efficient

Flash Error Model (Channel)

In this project, the communication/storage channel is not modeled as a simple independent random bit-flip channel. Instead, the evaluator simulates several error behaviors inspired by real NAND Flash reliability issues. These error patterns are intended to reflect the fact that Flash memory errors are often structured, localized, time-dependent, and non-uniform.

Your ECC system will therefore be evaluated not only on its ability to handle random noise, but also on its robustness under more realistic corruption patterns, including burst errors, retention-related degradation, and disturb-like clustered corruption.

The exact corruption pattern of each hidden test case will not be revealed in advance.

Error Types Used in Evaluation

1. **Random Bit Errors**

   The evaluator may apply sparse random bit flips across the encoded codeword. These errors simulate baseline noise, imperfect sensing, or mild instability in Flash cells.

   Characteristics:

   - errors are scattered rather than clustered
   - severity is usually low to moderate
   - can appear alone or together with other error types

   This is the simplest corruption pattern in the project.

2. **Burst Errors**

   The evaluator may corrupt several consecutive bytes within a local region. This simulates localized failure, page-level instability, or region-specific degradation.

   Example:

```
Original  : A B C D E F G
Corrupted: A X X X E F G
```

Characteristics:

- affects consecutive bytes
- may be short or moderately long
- is especially challenging for ECC schemes that assume independent random errors

3. **Retention Errors**

   Retention errors are aging-related errors whose overall severity increases with T. As data remains stored longer in Flash memory, charge leakage may gradually alter the stored state and increase the probability of readout errors.

   In this project, retention effects are modeled through the aging parameter T and the reference severity function $\hat{p}(T)$, with bounded variation around it.

4. **Disturb-like Cluster Errors**

   The evaluator may flip multiple bits/bytes concentrated in a small neighborhood of the codeword, simulating cell-to-cell program/read disturb in densely-packed Flash arrays.

   Characteristics:

   - Errors fall within a localized cluster of typically a few bytes to a few tens of bytes.
   - The cluster may overlap with random or burst errors.
   - Differs from a burst in that bits inside the cluster are flipped probabilistically rather than every consecutive byte.

Aging Parameter T

Each test case is associated with a non-negative integer parameter T, where:

$$1 \leq T \leq 1000$$

T represents the storage-aging level of the encoded data. A larger T means that the stored codeword is considered older and therefore more vulnerable to retention-related corruption.

In this project, T is an abstract aging index, not a physical unit of time such as seconds, hours, or days. It is introduced to model the fact that Flash memory reliability degrades as data remains stored for longer periods.

Both the encoder and decoder will receive the same value of T through the command-line interface.

Reference Retention Severity Function

To provide a predictable but non-trivial aging model, the evaluator uses the following reference retention-related corruption severity function:

$$p(T) \in [(1-\epsilon)\,\hat{p}(T),\ (1+\epsilon)\,\hat{p}(T)], \quad \hat{p}(T) = 0.06\sqrt{\frac{T}{1000}}, \quad \epsilon = 0.1$$

Therefore, the actual retention-related severity may vary by up to ±10% relative error around the reference value.

For example, if T = 1000, then:

$$\hat{p}(1000) = 0.06$$

and the actual severity may fall within:

$$[0.054, 0.066]$$

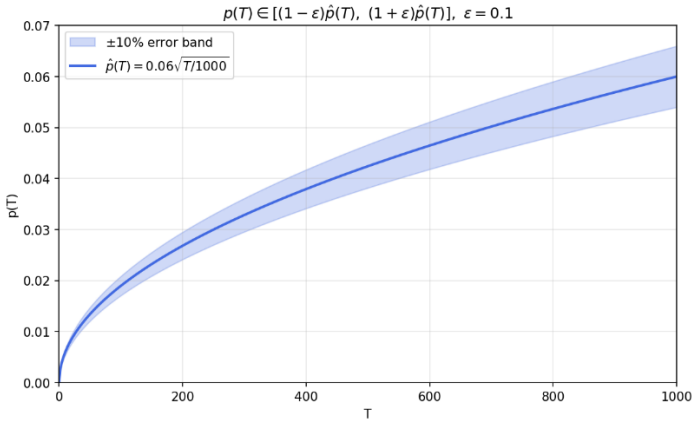
This design is intended to reflect natural variation in Flash reliability and to prevent overfitting to a single exact formula.

The value $p(T)$should not necessarily be interpreted as an exact independent bit-flip probability applied uniformly to every bit. Instead, it should be understood as an approximate measure of retention-related corruption intensity.

In some test cases, the evaluator may use $p(T)$to determine:

- how many bits or bytes are likely to become vulnerable,
- how large a damaged region may be,
- how likely a local cluster of corrupted bytes may occur,
- or how strongly retention-related effects are combined with other Flash-inspired error types.

Therefore, your system should aim to be robust under aging-related degradation, not merely tuned to a single exact stochastic process.



Program Interface

1. Required Executable
   Your submission must provide one executable named solution, which supports the following two modes:

   ```
   ./solution encode T <input_file> <output_file>
   ./solution decode T <input_file> <output_file>
   ```

   Here, T represents the storage time (or aging time) associated with the data. Larger T indicates that the data has been stored for a longer period and may therefore be subject to a higher retention-related error rate in the evaluation process.

2. Meaning of T
   T is a non-negative integer provided by the evaluator. It is part of the storage model and should be treated as an input parameter to your encoding and decoding strategy.

   You may use T in any way you consider appropriate. For example, your system may choose different redundancy levels, block organizations, or protection strategies for different values of T.

   However, the evaluator will still apply hidden corruption models, and using T does not reveal the exact corruption pattern.

3. Encoder Input
   When running in encode mode

   ```
   ./solution encode T <input_file> <output_file>
   ```

   - T: storage time parameter

- input_file: file containing the original message
- output_file: file to which the encoded codeword must be written

The input_file format is:

- Line 1: integer N — original message length in bytes
- Line 2: hexadecimal string of length 2N, representing the original message bytes

Example of input_file:

```
8
0123456789ABCDEF
```

The program must read from input_file and write the encoded result to output_file.

4. Encoder Output
   The encoder must write exactly one line to output_file:

   - A hexadecimal string representing the encoded codeword c

   The output must satisfy all of the following:

   - Contains only valid hexadecimal characters (0-9, A-F)
   - Has even length
   - Represents the full encoded codeword

   No extra spaces, explanations, or debug messages may be written to output_file.

5. Decoder Input
   When running in decode mode:

```
./solution decode T <input_file> <output_file>
```

   - T: storage time parameter
   - input_file: file containing the corrupted codeword
   - output_file: file to which the reconstructed message must be written

   The input_file format is:

   - Line 1: integer N — original message length in bytes

- Line 2: hexadecimal string representing the corrupted codeword c'

The decoder is given the original message length N and the same storage-time parameter T, but not the original message content.

6. Decoder Output

The decoder must write exactly one line to output_file:

- A hexadecimal string of length 2N, representing the reconstructed message m'

The output is considered correct if and only if the reconstructed message is byte-by-byte identical to the original message.

No extra spaces, explanations, or debug messages may be written to output_file.

Baseline

Testcase	Correctness	Testcase	Correctness
Public1	100%	Public2	100%
Public3	90%	Public4	60%

Scoring

Overall Rule

Each hidden case is worth **10 points** in total.

For each hidden case, the evaluator will test the submission on all:

$$T = 1,2, \dots, T_{\max}$$

and compute three quantities:

- **correctness**
- **runtime**
- **redundancy**

Their weights are:

- correctness: **50%**
- runtime: **25%**
- redundancy: **25%**

Thus, for each hidden case:

- correctness contributes **5 points**
- runtime contributes **2.5 points**
- redundancy contributes **2.5 points**

Correctness

For one hidden case, correctness is defined as the fraction of aging levels

$$T = 1,2, \dots, T_{\max}$$

where T_{max}denotes the maximum T value observed in the given testcase, for which the submission successfully reconstructs the original data exactly. Let self(corr) be the correctness of the current submission on this hidden case, avg(corr) be the average correctness over all submissions, and max(corr) be the maximum correctness over all submissions.

The correctness score is defined as:

$$\text{Correctness Score} = 2.5 + 2.5 \times \max\left(\frac{\text{self(corr)} - \text{avg(corr)}}{\text{max(corr)} - \text{avg(corr)}}, -1\right)$$

This score is clipped to the range [0, 5].

If the denominator is 0, then all submissions receive 2.5 points for this part.

Runtime

For one hidden case, runtime is defined as the total runtime of all encoder and decoder calls over

$$T = 1,2, \dots, T_{\max}$$

Let self(runtime) be the total runtime of the current submission on this hidden case, avg(runtime) be the average runtime over all submissions, and min(runtime) be the minimum runtime over all submissions.

The runtime score is defined as:

$$\text{Runtime Score} = 1.25 + 1.25 \times \max\left(\frac{\text{avg(runtime)} - \text{self(runtime)}}{\text{avg(runtime)} - \text{min(runtime)}}, -1\right)$$

This score is clipped to the range [0, 2.5].

If the denominator is 0, then all submissions receive 1.25 points for this part.

Redundancy

For each value of T, let $|\ m\ |$ be the original message length and $|\ c_T\ |$ be the encoded length. The redundancy ratio at age T is

$$\frac{|\ c_T\ |}{|\ m\ |}$$

For one hidden case, redundancy is defined as the average of this ratio over all

$$T = 1,2, \dots, T_{\max}$$

That is, let self(red) be the average redundancy ratio of the current submission, avg(red) be the average over all submissions, and min(red) be the minimum over all submissions.

The redundancy score is defined as:

$$\text{Redundancy Score} = 1.25 + 1.25 \times \max\left(\frac{\text{avg(red)} - \text{self(red)}}{\text{avg(red)} - \text{min(red)}}, -1\right)$$

This score is clipped to the range [0, 2.5].

If the denominator is 0, then all submissions receive 1.25 points for this part.

Correctness Gate

Correctness is the most important metric in this project.

If the correctness of a submission on a hidden case is lower than the average correctness by more than one standard deviation, i.e.,

$$\text{self(corr)} < \text{avg(corr)} - \sigma(\text{corr}),$$

then the submission receives no runtime score and no redundancy score for that hidden case.
In that case:

- Runtime Score = 0
- Redundancy Score = 0

Maximum Input Size

Each test case contains at most 512 bytes of original data.
If the input size exceeds this limit, that test case will not be used in the official evaluation.

Hard cap on Encoding length

For every value of T, the encoded output must satisfy

$$| c_T | \leq 2 | m |$$

If this limit is violated for any T, that run is treated as invalid and counted as a failed trial.

Timelimit

Each individual call to encode or decode has a time limit of 1 second.
If a call exceeds the time limit, that run is treated as failed.

Single Case Score

For each hidden case, the total score is

$$\text{Hidden Case Score} = \text{Correctness Score} + \text{Runtime Score} + \text{Redundancy Score}$$

The full score of one hidden case is 10 points.

Additional Note

- All hidden cases will be evaluated on every
  $$T = 1,2, \dots, T_{\max}$$
  without sampling.
- Correctness is the primary objective of this project.
- Runtime and redundancy are only meaningful if the solution is sufficiently reliable.
- The program must not read from or write to any files other than the specified input_file and output_file.
- The solution must be deterministic. Randomized behavior without fixed seeds may lead to unstable results.

Testcases

For each test case, there is only one input.txt and one meta.txt describing the information of that test case. In addition, for the same test case, the input.txt is identical for all values of T.

Folder:

```
testcases/
├── public1/
│   ├── input.txt
│   └── meta.txt
├── public2/
├── public3/
└── public4/
```

Input:

```
8
0123456789ABCDEF
```

Meta:

```
id=example
T_max=5
msg_size=8
seed=3237998081
```

Scripts

Two helper utilities are provided so you can reproduce the grading flow on your own machine.

evaluator

Usage:

```
./evaluator <solution_path> <testcase_dir> [--quiet]
```

Runs your solution on a single test case. For every T in 1..T_max the evaluator

1. invokes <solution_path> encode T to obtain a codeword c,
2. corrupts c with the Flash channel model described in Section 2,
3. invokes <solution_path> decode T to recover m',
4. compares m' against m and records correctness, runtime, and |c| / |m|.

testcase_dir must contain input.txt and meta.txt. The evaluator prints a per-T pass/fail line followed by a summary block; pass --quiet to suppress the per-T lines.

Examples:



A trial fails (and is excluded from correct_count) if the solution exits non-zero, exceeds the 1-second time limit, violates the |c| <= 2|m| cap, or emits a malformed output file.

grade.sh

Usage:

```
./grade.sh <submissions_dir> <testcase_dir>
```

Batch driver: for every *.tar.gz under submissions_dir, the script

1. extracts the archive into a fresh temp directory,
2. locates the project root (preferring <StudentID>_final/),
3. removes any pre-built bin/solution, then runs make,
4. invokes evaluator against every immediate subdirectory of testcase_dir that contains input.txt and meta.txt,
5. appends one row per (student, case) to grades.csv (long format) next to the script.

The script does not abort on a single failed submission; build failures or missing files are logged in the error column of grades.csv.

You can use this on your own submission before uploading to verify the archive is well-formed and that make produces bin/solution on a clean checkout:

```
mkdir self_test
cp <StudentID>_final.tar.gz self_test/
./grade.sh self_test ./testcases
cat grades.csv
```

If your row contains an error (for example "make failed" or "bin/solution missing") the grader will see the same problem on your real submission.

Examples:

```
[ta114062584@linuxcad30 ~/student]$ ./grade.sh ../test_submissions testcases
grading 2 submission(s) against 4 case(s) -> /home/HardwareSecurity_1142/ta114062584/student/grades.csv
cases: public1 public2 public3 public4

[1/2] 888888_final.tar.gz
  public1      corr=100.00% rt=392.8ms red=2.0000
  public2      corr=100.00% rt=1519.8ms red=1.9844
  public3      corr=91.47% rt=6527.9ms red=1.9844
  public4      corr=53.60% rt=9803.1ms red=1.9844

[2/2] 999999_final.tar.gz
  public1      corr=100.00% rt=463.1ms red=1.4650
  public2      corr=100.00% rt=1531.6ms red=1.4344
  public3      corr=95.73% rt=6240.0ms red=1.7443
  public4      corr=73.30% rt=10339.4ms red=1.8043

done -> /home/HardwareSecurity_1142/ta114062584/student/grades.csv
```

Report

Your report must contain the following contents, and you can add more as you wish.

1. Names and student IDs.
2. Overview of your ECC system design.
3. Explanation of the main algorithm(s) used in your design.
4. How your design handles the given Flash error model and aging parameter T.
5. Experimental results, including correctness, runtime, and redundancy when appropriate.
6. Important implementation details. DO NOT paste all the code in the report, but you may include necessary code snippets, figures, or formulas.
7. What you learned from this project, and what problems you encountered.
8. Suggestions and feedback for this project.

Submission Requirement

You may use any programming language to implement this program. The use of external libraries is strictly prohibited. Any form of plagiarism is strictly prohibited.

Compress all the item with the name {StudentID}_final.tar.gz before uploading it to eeclass. And the folder structure would be like the following. Otherwise, your program will not be graded.

Example:

```
{StudentID}_final/
├── bin/
│   └── solution
├── src/
│   ├── main.cpp
│   └── utils.cpp
├── include/
│   └── utils.h
├── Makefile
└StudentID_report.pdf
```

Before evaluation, the make command will be executed in the root directory. The executable file should be generated at ./bin/solution, and the program will be executed using this file.

You can use the following command to compress your directory on a workstation:

$ tar -zcvf {StudentID}_final.tar.gz <directory>

For example:

$ tar -zcvf 114062584.tar.gz 114062584_final/

Grading

The grading is as follows:

Step 1 (40%): Public Cases

The evaluator will provide **4 public cases**, and each public case is worth **10 points**.

For each public case, a submission receives the full **10 points** if it reaches the required **correctness baseline** announced for that case.

Step 2 (40%): Hidden Cases

The evaluator will use **4 hidden cases**, and each hidden case is worth **10 points**.

Each hidden case will be scored according to the official hidden-case scoring rule described in the specification.

Step 3 (20%): Report

The report will be graded based on the following aspects:

1. **System Design and Algorithm Description (5%)**
   Clearly explain the overall ECC system design, including the main idea of the encoder and decoder.
2. **Error-Model Handling (5%)**
   Explain how the proposed method handles the given error model, such as random errors, burst errors, retention errors, and disturb-like errors.
3. **Experimental Results and Analysis (5%)**
   Present the results on public cases and/or self-designed test cases, and discuss correctness, runtime, redundancy, and the trade-offs among them.
4. **Clarity, Organization, and Reflection (5%)**
   The report should be well organized and readable, and should include implementation details, difficulties encountered, what was learned from the project, and suggestions or feedback.

Bonus (up to 10 points): Hidden-Case Ranking Bonus

For each hidden case, the top 5 submissions will receive **+5 bonus points**.

Each student may receive bonus points from at most **two hidden cases**, so the maximum bonus is **+10 points**.

No late submissions will be accepted for the final project; please submit it on time.

Contact

For any questions regarding this final project, please first post them on the discussion board on EEClass.

References

Reed–Solomon error correction - Wikipedia